

to add LLM-aware instrumentation. It helps capture model invocations, prompt and completion metadata, token usage, retries, errors, and interactions with frameworks such as LangChain, LlamaIndex, and related libraries.

Its significance goes beyond convenience. OpenLLMetry aligns AI telemetry with emerging semantic conventions for generative AI, making it easier to move data between observability backends without rewriting instrumentation.

OpenInference and Arize

Phoenix: OpenInference provides a complementary set of conventions and instrumentation patterns for AI application tracing. It is designed to describe LLM calls, retrieval events, embeddings, tool usage, and surrounding application context in a consistent format. This is especially helpful in complex systems where teams want observability data to preserve meaning rather than exist as generic spans with unclear attributes. Clear conventions improve downstream analysis because they make traces easier to compare, filter, and evaluate.

Arize Phoenix is widely used as an open source platform for inspecting and debugging LLM applications, especially RAG systems. It offers trace visualisation, dataset analysis,

evaluation support, and workflow inspection in a way that is approachable for both engineers and applied AI teams. One of its strengths is that it helps teams move from raw telemetry to interpretability. Instead of only seeing that a request failed or slowed down, users can inspect retrieval context, compare outputs, study response patterns, and investigate why a chain or agent behaved the way it did.

Langfuse: Langfuse is one of the best-known open source platforms built specifically for LLM engineering workflows. It combines tracing, prompt observation, version tracking, usage analytics, and evaluation-oriented features in a single environment. For teams building chatbots, copilots, search assistants, or multi-step chains, Langfuse offers a practical way to inspect how prompts evolve, how model responses vary across versions, and where latency or cost accumulates along a workflow.

Helicone: Helicone takes a different approach by acting as a proxy layer for model traffic. This design is useful when teams want fast visibility with minimal code change. By routing LLM requests through a proxy, organisations can capture usage, latency, cost, and response metadata across providers without deeply modifying application

logic. That makes it attractive in environments where speed of adoption matters or where multiple teams share common AI gateways. Proxy-based observability is not always as expressive as deep in-process instrumentation, but it can provide a practical and low-friction starting point for gaining operational insight into production traffic.

MLflow: MLflow is traditionally associated with machine learning lifecycle management, but it remains relevant in AI and LLM observability because experiment tracking and model governance are closely tied to production behaviour. In LLM-centric workloads, MLflow can help record prompt variants, evaluation outcomes, parameter settings, model versions, and deployment metadata. While it is not a specialised tracing tool in the same way as Langfuse or Phoenix, it provides a valuable layer of reproducibility. Teams that want to compare model behaviour across experiments, retain lineage, and connect offline evaluation with online performance often find MLflow useful as part of a broader observability stack.

TruLens: TruLens is especially valuable when the main question is not just how the system performed, but whether the response was actually good. It brings an evaluation-centric lens to observability by helping teams assess groundedness, relevance, and output quality in RAG and LLM applications. This makes it well suited to environments where correctness and trust matter more than raw throughput. In practice, tools such as TruLens fill an important gap: conventional telemetry can reveal that a request completed successfully, yet still say nothing about whether the answer was useful, accurate, or aligned with expectations.

OpenLIT: OpenLIT has emerged as another notable open source option for monitoring LLM applications and

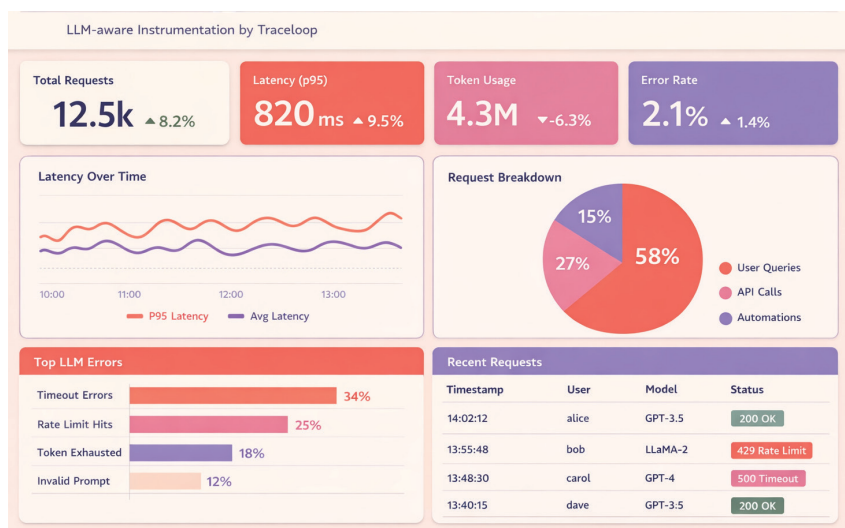


Figure 2: OpenLLMetry LLM monitoring dashboard